

Little Gerry — AI Executive Assistant for Precisian Medical Instruments

Little Gerry is the AI Executive Assistant, chief of staff, and regulatory expert for [Precisian Medical Instruments](#) and the **VACTOR** device program. It combines a powerful cloud AI brain (Anthropic Claude) with local-first data storage — your documents and conversations stay on your machine, while your AI is best-in-class.

Table of Contents

1. [Overview](#)
 2. [Features](#)
 3. [Architecture](#)
 4. [Windows Installer \(Recommended\)](#)
 5. [Prerequisites \(Manual Setup\)](#)
 6. [Setup & Installation \(Manual\)](#)
 7. [AI Configuration](#)
 8. [Running the Application](#)
 9. [Usage Guide](#)
 10. [Configuration Reference](#)
 11. [Development](#)
 12. [Security Notes](#)
-

Overview

Little Gerry runs as a **native Windows desktop application** — a branded splash screen launches all services in the background, then loads the full React app in a WebView2 window. A system tray icon provides quick access and graceful shutdown.

The stack: **React 19 + TypeScript + Vite** frontend, **FastAPI** backend (Python 3.14), **Anthropic Claude** for AI, **Voyage AI** for document embeddings, and **PostgreSQL 16 + pgvector** for vector search — all data stored locally on your machine.

Key design principles:

- **Local-first data, cloud-flexible inference** — all documents, conversations, and tasks are stored on your machine; AI inference is provided by Anthropic (or OpenAI / Ollama optionally)
- **Medical-grade context** — purpose-built prompts for FDA/ISO regulatory workflows, CAPA, and VACTOR device documentation
- **Human-in-the-loop** — AI suggestions require explicit approval before executing consequential actions
- **Full audit trail** — every AI action and document change is logged immutably

Features

Module	Description
AI Chat (Little Gerry)	Streaming conversational AI with tool use, RAG over your knowledge base, and real-time WebSocket responses
Ask Gerry about this	One-click button on any task, project, contact, email, draft, calendar event, KB document, generated file, or attachment — opens a chat seeded with that item (files are read in full)
Dashboard	At-a-glance view of tasks, pending approvals, today's meetings, and AI-generated daily briefing
Daily Assistant	Once-a-day background scan of your Gmail and Google Tasks that surfaces suggested follow-ups and to-dos for human review (sidebar badge shows pending count)
Projects & Tasks	Kanban board with drag-and-drop, project tracking, due dates, and priority management
Calendar	Scheduled events with meeting integration
Knowledge Base	Upload and semantically search internal documents (PDFs, DOCX, TXT); auto-chunked and embedded
Search	Semantic vector search across all uploaded documents with category filtering
Research	AI-assisted literature/regulatory research with cited responses

Module	Description
Meeting Notes	Auto-detects video calls (Zoom/Teams/Meet) and records + transcribes system audio, then AI-summarizes with one-click action item extraction → Tasks; manual transcript paste also supported
Gmail	Full inbox: read/search/tag emails, reply & reply-all, move to Trash, open attachments in Google Workspace, and compose & send your own email directly (no approval needed for mail you write yourself)
Email Drafts	AI-generated email drafts for regulatory, investor, and operational communications; submit for approval to send from your Gmail account
Regulatory	File explorer for regulatory documents (DHF, IFU, 510(k), ISO 13485): browse/create folders, upload, import from Drive, edit, rename, move, and delete — write access gated per user
Investor Relations	IR hub: company snapshot, regulatory proof-points, AI-drafted pitch context, research feed, and IR specialist chat
Approvals	Workflow approval queue — approve/reject with automatic execution and full audit trail
Notifications	Real-time WebSocket push notifications with read/unread management
Feedback	Top-bar button to report a bug or request a feature; submissions are routed to the owner's notifications
Setup Wizard	One-time guided first-use onboarding: explains the stack, connects Claude + Voyage (pre-set defaults) and Google, and covers roles & usage
Audit Trail	Immutable log of all system and AI actions with filtering and export
User Management	Google sign-in only (no passwords); invite teammates by email; accounts auto-created on first sign-in (owner = admin, everyone else = full-access member); per-user Regulatory write permission and deactivation
Google Workspace	Connect your Google account for Gmail, Drive, Calendar, and Contacts integration with human-in-the-loop write approvals

Module	Description
Odoo ERP	Connect an Odoo account via encrypted API key; Little Gerry reads ERP data and proposes write actions through the Approvals queue
Settings	LLM model selection, embedding provider, re-index KB, live health monitoring, appearance, notification preferences, and one-click in-app updates

Architecture



Embedding dimensions are provider-native. Each embedding provider returns its own vector size. Switching providers requires re-indexing the Knowledge Base (Settings → AI Engine → Re-index Now).

Windows Installer (Recommended)

Requirements

- Windows 10 or 11 (64-bit)
- Internet access (~3–5 GB download on first run)
- ~5 GB free disk space

Steps

1. **Build the installer** (skip if you have a pre-built `LittleGerry_Setup.exe`):

```
Double-click: build-installer.bat
```

Output: `installer\Output\LittleGerry_Setup.exe`

2. **Run `LittleGerry_Setup.exe`** — the installer will:
 - Install Docker Desktop, Ollama, Python 3.14, and Node.js (via winget, if not already present)
 - Copy all application files to `C:\Users\<you>\AppData\Local\LittleGerry\`
 - Create a desktop shortcut and Start Menu entry
3. **Launch** via the desktop shortcut or `Start LittleGerry.bat`.

On first launch, a setup wizard runs automatically and asks you to enter:

- Your company email address (used as your login)
- Your display name

- A password of your choice

It then starts PostgreSQL, runs database migrations, seeds your account, and installs frontend dependencies before launching Little Gerry.

Install location: `C:\Users\<you>\AppData\Local\Little Gerry\`

Utility Scripts

File	Purpose
<code>Start Little Gerry.bat</code>	Start all services and launch Little Gerry (runs first-time setup if needed)
<code>Stop Little Gerry.bat</code>	Gracefully stop backend, frontend, and PostgreSQL
<code>Update Little Gerry.bat</code>	Pull the latest version from GitHub and restart
<code>Install Little Gerry.bat</code>	Re-run the full setup (useful after a clean clone)
<code>build-installer.bat</code>	Compile <code>LittleGerry_Setup.exe</code> using Inno Setup 6

Updating

Go to **Settings** → **Updates** and click **Check for Updates**. If a new version is available, click **Install Update** — the app pulls from GitHub, runs migrations, and restarts automatically.

You can also run `Update Little Gerry.bat` directly at any time. The update script automatically ensures Docker and PostgreSQL are running before applying database migrations.

Exit Confirmation

Closing the Little Gerry window shows a native confirmation dialog before shutting down all services. Using **Stop All Services** from the system tray bypasses the dialog (the tray action is already intentional).

Prerequisites (Manual Setup)

Requirement	Version	Notes
Windows 10/11	64-bit	Primary supported platform
macOS	12+ (Apple Silicon)	Supported — see the macOS section
Node.js	v20+	nodejs.org
Python	3.14	python.org
uv	latest	<code>pip install uv</code>
Docker Desktop	latest	docker.com
Anthropic API key	—	console.anthropic.com — required for AI chat
Voyage AI API key	—	dash.voyageai.com — free tier, required for KB/search
Ollama (optional)	latest	ollama.com — only needed if using Ollama as LLM or embedding provider
Inno Setup 6	6.x	Only needed to build the installer

Setup & Installation (Manual)

1. Clone the repository

```
git clone https://github.com/soulenya/pmi-agent.git
cd pmi-agent
```

2. Start PostgreSQL (Docker)

```
docker compose up -d
```

Starts a `pgvector/pgvector:pg16` container named `pmi_postgres` on port **5432**.

3. Set up the Python backend

```
cd backend
uv sync
cp .env.example .env
uv run alembic upgrade head
uv run python scripts/seed_admin.py
```

`seed_admin.py` reads `PMI_ADMIN_EMAIL`, `PMI_ADMIN_NAME`, and `PMI_ADMIN_PASSWORD` from environment variables, or prompts interactively.

4. Set up the frontend

```
cd frontend
npm install
```

Create `frontend/.env`:

```
VITE_API_BASE=http://127.0.0.1:8000
VITE_WS_BASE=ws://127.0.0.1:8000
```

Running the Application

Desktop app (recommended)

```
# From the project root:
backend\.venv\Scripts\pythonw.exe launcher.py
```


`launcher.py` opens a branded splash screen in a pywebview WinForms window, starts all services in the background, then navigates to the React app when services are ready.

Manual / developer mode

Terminal 1 — Backend

```
cd backend
uv run uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Terminal 2 — Frontend

```
cd frontend
npm run dev
```

Open **`http://localhost:5173`** in your browser.

macOS (Apple Silicon)

Little Gerry runs on macOS using the same backend, frontend, and Docker stack. The launcher (`launcher.py`) is cross-platform; macOS-specific scripts live alongside the Windows `.bat` files.

First-time setup

```
git clone https://github.com/soulenya/pmi-agent.git
cd pmi-agent
bash scripts/install.sh          # installs Homebrew deps, DB, migrations, frontend
```

`install.sh` installs the prerequisites via Homebrew (Docker Desktop, Python, Node 20, `uv`), brings up PostgreSQL, runs migrations, seeds the admin user, and makes the launcher scripts executable.

Running

Double-click **Start Little Gerry.command** in Finder (or run it from a terminal). It performs first-run setup if needed, then opens the app in a native Cocoa/WebKit window. **Stop Little Gerry.command** shuts the services down.

```
If a script won't run after a fresh git clone, make them executable once: chmod +x *.command scripts/*.sh
```

File	Purpose
<code>Start Little Gerry.command</code>	Start all services and launch Little Gerry (first-run setup if needed)
<code>Stop Little Gerry.command</code>	Gracefully stop backend, frontend, and PostgreSQL
<code>scripts/install.sh</code>	Full Homebrew-based setup (run once after cloning)
<code>scripts/update.sh</code>	Developer update — git pull, deps, migrations, restart
<code>scripts/build-macos.sh</code>	Build the macOS installer <code>installer/Output/LittleGerry.pkg</code>
<code>scripts/publish-macos.sh</code>	Build and attach <code>LittleGerry.pkg</code> to the GitHub release

Building the macOS installer (.pkg)

```
bash scripts/build-macos.sh
```

Produces a per-user `installer/Output/LittleGerry.pkg` that installs to `~/Applications/Little Gerry` with a double-clickable **Little Gerry.app** launcher — the macOS analog of the Windows `LittleGerry_Setup.exe`.

Code signing / notarization (requires a paid Apple Developer account) is optional and enabled via environment variables — when unset, the build is unsigned (works on your own Mac; other Macs need a one-time right-click → Open):

```
export DEVELOPER_ID_INSTALLER="Developer ID Installer: Your Name (TEAMID)"
export NOTARY_PROFILE="littlegerry" # from: xcrun notarytool store-credentials
```

```
bash scripts/publish-macos.sh  
release
```

```
# builds, signs, notarizes, uploads to the
```

The Windows **.exe** and macOS **.pkg** attach to the **same** GitHub release/tag, and the in-app auto-updater downloads the correct asset for each platform automatically.

Usage Guide

Logging In

Click **Sign in with Google**. Little Gerry uses Google sign-in only — there are no passwords to manage. Your account must belong to an approved Workspace domain (**pmi-llc.com** or **precisianmedical.com**). On your **first** sign-in your account is created automatically: the owner becomes the **admin**, and everyone else joins as a **full-access member**. The one-time Setup Wizard then walks you through configuration.

Inviting Teammates

Admins can invite others from **Users** → **Invite**:

1. Enter the teammate's **email** (optionally a display name and a personal note).
2. They receive an email with a link to download Little Gerry and instructions to **Sign in with Google**.
3. Their account is created automatically on first sign-in as a **full-access member** — no passwords, no manual setup.

Because each person runs their own local copy, every install keeps its own data; the invite simply gets them the app and an account on their own machine.

AI Chat — Little Gerry

Navigate to **AI Assistant** in the sidebar.

- Type a message and press **Enter** (Shift+Enter for a new line)

- Little Gerry can search your knowledge base, look up tasks, summarize documents, and draft content
- Rename or archive conversations by hovering over them in the left panel
- Click **+** for a new conversation
- Click **Ask Gerry about this** on any task, email, file, contact, or event to open a chat already seeded with that item (attachments and generated files are read in full)

Example prompts:

Summarize the VACTOR DHF status and flag any gaps
What are my open tasks this week?
Draft an email to the FDA about our PMA submission timeline
Generate a risk assessment section for the VACTOR IFU

Knowledge Base & Search

1. Navigate to **Knowledge Base** → **Upload Document** → select a PDF, DOCX, or TXT file and assign a category
2. The document is automatically chunked, embedded (using your configured embedding provider), and indexed
3. Navigate to **Search** to query with natural language — results are ranked by semantic similarity

Embedding provider must be configured first. Go to Settings → AI Engine and enter your Voyage AI API key (free at dash.voyageai.com) before importing documents. Voyage AI is the recommended provider and stores vectors at native 1024 dimensions for superior retrieval quality.

Switching embedding providers after importing documents requires re-indexing. Go to Settings → AI Engine — if the new provider uses a different vector dimension, a warning will appear and you must click **Re-index Now** before semantic search will work. The re-index re-embeds all documents through the new provider.

Research

Navigate to **Research** → **New Research Report** → enter a topic. Little Gerry searches the web (DuckDuckGo), reads sources, and generates a structured cited report stored in the research archive.

Calendar

Displays local events and Google Calendar events side by side. Use the **Sync** button (top right, visible when Google is connected) to refresh Google Calendar on demand.

Tasks & Projects

- **Tasks** — Kanban board (To Do → In Progress → In Review → Done)
 - **Projects** — Group tasks by project with milestone tracking
-

Gmail

Open **Gmail** from the Communications area (Google must be connected). Read, search, and tag-filter your inbox; reply or reply-all; move messages to Trash; and open attachments in Google Workspace. Click **New email** → **Write it myself** to compose and send email directly from your Gmail account (no approval needed for mail you write yourself). You can also ask Little Gerry to draft replies, which route to Approvals.

Email Drafts

Navigate to **Email Drafts** → **New Email Draft** → fill in topic and context → **Generate**. Review, then submit for approval to send from your Gmail account. Errors are shown inline if generation fails.

Meeting Notes

Little Gerry can **auto-record and transcribe video calls** (Zoom/Teams/Meet) when auto-recording is enabled in Settings (system-audio capture is Windows-only), then

summarize them automatically. To add one manually:

1. **+ New Meeting** → paste or type the meeting transcript
 2. **Summarize** — generates a structured summary with decisions and action items
 3. **Extract Actions** → select items → **Create N Tasks** — adds directly to the Kanban board
-

Regulatory Documents

Hover over a document row → click **AI Draft** — Little Gerry retrieves relevant knowledge base context and generates professional regulatory content. Review and edit before use.

Google Workspace

Navigate to **Google Workspace** in the sidebar.

1. Click **Connect Google** — your default browser opens Google's sign-in page
2. Sign in with your Google account and grant the requested permissions
3. Once connected, the status turns green and your Google account email is shown

Connected services: Gmail (read + send), Google Drive (read), Google Calendar (read + create events), and Contacts (read).

Write actions (send email, create calendar event) go through the human-in-the-loop approval queue on the same page — click **Approve** or **Cancel** before they execute.

To disconnect: click **Disconnect Google**. Your local token is deleted immediately.

Approvals

Navigate to **Approvals** for the human-in-the-loop queue. Pending approvals show AI-generated summaries. Click **Approve** or **Reject** with an optional comment.

Settings

- **Profile** — update display name and password
- **AI Engine** — LLM provider + model + API key; Embedding provider + model + API key; Re-index Knowledge Base button (appears when switching providers that use a different vector dimension); live System Health panel showing LLM • and Embedding • status with actual API ping results
- **Appearance** — Light, Dark, or System theme; Timezone
- **System Health** — live status of PostgreSQL, active LLM (with live ping), active embedding provider (with live ping), disk space, and re-index flag
- **Updates** — check and install updates in-app

Automatic updates on launch. Each time Little Gerry starts it checks GitHub for a newer version. **Installed apps** download the latest signed installer and apply it in the background (stop → silent install → relaunch) — the updater runs as an independent process so it completes even after the app closes to swap files. **Developer checkouts** instead pull the latest code, refresh dependencies, and apply pending database migrations. Either way, the in-app **Updates** panel remains available for an on-demand check. (Auto-update is skipped on developer machines with uncommitted changes.)

Configuration Reference

Backend — `backend/.env`

```
DATABASE_URL=postgresql+asyncpg://pmi_app:pmi_dev_password@localhost:5432/pmi_dev
DATABASE_URL_SYNC=postgresql://pmi_app:pmi_dev_password@localhost:5432/pmi_dev
DEFAULT_LLM_PROVIDER=anthropic
DEFAULT_LLM_MODEL=claude-sonnet-4-6
DEFAULT_EMBEDDING_PROVIDER=voyage
DEFAULT_EMBEDDING_MODEL=voyage-3
DEFAULT_EMBEDDING_DIMENSION=1024
OLLAMA_BASE_URL=http://localhost:11434
DEBUG=false
```

API keys (Anthropic, OpenAI, Voyage AI) are **never** stored in `.env` — they are entered in Settings → AI Engine and stored in the OS keychain (Windows Credential Manager).

System Health API

Two health endpoints are available:

Endpoint	Purpose
<code>GET /health</code>	Full system check: database, LLM live ping, embedding live ping, disk space, re-index flag
<code>GET /settings/health</code>	Lightweight AI-only check: LLM + embedding live pings only (< 3s), used by Settings page

Sample `GET /health` response:

```
{
  "status": "ok",
  "checks": {
    "database": {"status": "ok"},
    "llm": {"status": "ok", "provider": "anthropic", "model": "claude-sonnet-4-6"},
    "embedding": {"status": "ok", "provider": "voyage", "model": "voyage-3",
    "dimension": 1024},
    "kb_needs_reindex": false,
    "disk": {"status": "ok", "free_gb": 42.5}
  }
}
```

Frontend — frontend/.env.development

```
VITE_API_BASE=http://127.0.0.1:8000
VITE_WS_BASE=ws://127.0.0.1:8000
```

Google Workspace

OAuth credentials are stored in `backend/google_credentials.json` (Desktop app type) and are **gitignored** — never committed to source control and **not bundled in the installers**. Add the file once after installing (see `docs/INSTALL.md` → "Google OAuth credentials"); it survives app updates. The per-user token is written to `backend/google_token.json` after first sign-in and is also gitignored. If you ever rotate the OAuth client secret in Google Cloud, distribute the new file privately.

Development

Backend type-checking

```
cd backend && uv run mypy .
```

Frontend type-checking

```
cd frontend && npx tsc --noEmit
```

Run backend tests

```
cd backend && uv run pytest
```

Database migrations

```
cd backend
uv run alembic revision --autogenerate -m "description" # new migration
uv run alembic upgrade head # apply
uv run alembic downgrade -1 # roll back one step
```

Build the Windows installer

Requires [Inno Setup 6](#).

```
Double-click: build-installer.bat
```

Output: `installer\Output\LittleGerry_Setup.exe`

Security Notes

- The backend binds to `127.0.0.1` only — not accessible from other machines on the network
- JWT secrets are stored in the OS keyring (Windows Credential Manager), not in `.env` files
- API keys (Anthropic, OpenAI, Voyage AI) are stored in the OS keychain via the `keyring` library — never written to disk
- All document files are encrypted at rest using Fernet symmetric encryption
- Rate limiting is applied to all API endpoints via `slowapi`
- CORS is restricted to localhost origins
- Google OAuth tokens are stored locally and never transmitted to any third-party service
- Sign-in is Google SSO only and restricted to approved Workspace domains (`pmi-llc.com`, `precisianmedical.com`); no passwords are stored or transmitted
- `backend/google_credentials.json` (OAuth client secret) is gitignored and is not bundled in the installers — users add it after installing (`docs/INSTALL.md`); rotate the secret in Google Cloud if it is ever exposed